

Project Overview

Title of the project is meme generator. As name suggests it generates a meme which is a combination of an image and text drawn over it. In this case text is in form of "{Quote} - {Author of the Quote}".

There are two interfaces to this project:

1) Command line: Takes Image File, Quote and Author as input from user and generates meme and returns the path of generated meme as output.

2) Web Interface: Takes Image File URL, Quote and Author as input from user, generates meme and displays that meme back to user in web interface.

In both the cases, if user fails to provide required input, it provides mechanism to pick random image and quote/author from local files and generate meme.

Components of the project

There are 4 broad components that forms the complete functionality built into the project.

- 1) app.py: This program provides web interface for meme generation.
- 2) meme.py: This program provides command line interface for meme generation.
- 3) QuoteEngine Module: This module comprises of different modules and programs that help parse different file types for Quotes to be displayed in the meme. These quotes are of the form "Body – Author". This module supports parsing of 4 different types of files – csv, docx, pdf and txt.
- 4) MemeEngine Module: This module comprises of programs that take input image, resize it and then draws supplied Quote on resized image turning the combination into a meme.

The project files layout is as under:

Source

| → app.py, meme.py, requirement.txt, README.pdf

| → QuoteEngine

 | → QuoteModel.py, IngestorInterface.py

 | → CSVIngestor.py, DocxIngestor.py, PDFIngestor.py, TextIngestor.py

 | → Ingestor.py

| → MemeEngine

 | → MemeEngine.py

- | → sw → Contains the 64 bit xpdf binaries required to provide PDF to text conversion functionality
- | → _data → Contains Local Image files and Quotes in different file formats
- | → templates → Contains different html forms to be used by app.py
- | → tmp → stores generated memes
- | → static → serves as root for flask app

Next we will cover each of the 4 components mentioned above that make up the project.

- 1) QuoteEngine Module : This module consists of seven modules
 - a) QuoteModel Class → This is implemented in QuoteModel.py and is a very simple class responsible for creating QuoteModel objects (with variables body and author). This class has no dependency on any other class.
 - b) IngestorInterface → This is implemented in IngestorInterface.py and is an abstract class defining components which are common to all File Type Ingestors. It defines two methods:
 - a. can_ingest method → This is a class method and determines whether or not a particular filetype can be ingested. It takes the extension of file supplied and checks if extension is contained in allowed_extensions class variable. If it is contained in allowed_extension it returns True else it returns False Boolean value.
 - b. parse method → This is an abstract and class method and not implemented. This method will be implemented by all File Type Ingestor classes that inherit IngestorInterface class. Basic functionality implemented will be it takes path of the file supplied (including filename); parses it for Quotes in the form of "body – author"; creates QuoteModel objects out of it (one object per quote) and stores those QuoteModel object in a list and then returns the list to calling program
 - c) CSVIngestor → This is implemented in CSVIngestor.py and inherits IngestorInterface abstract class and implements parse method for CSV File type.
 - d) DocxIngestor → This is implemented in DocxIngestor.py and inherits IngestorInterface class and implements parse method for DOCX file type. It uses docx module from python-docx package.
 - e) PDFIngestor → This is implemented in PDFIngestor.py and inherits IngestorInterface class and implements parse method for PDF file type. It uses pdftotext 64 bit binary from XPDF package for Windows utilizing subprocess module. This software is installed in src/sw.
 - f) TextIngestor → This is implemented in TextIngestor.py and inherits IngestorInterface class and implements parse method for TXT file type.
 - g) Ingestor → This is implemented in Ingestor.py and inherits IngestorInterface class and implements parse method using logic which selects correct Type Ingestor class to parse the supplied file and then calls the correct Type Ingestor Class to parse the file. app.py and meme.py makes use of this class to invoke correct Type Ingestor class.
- 2) MemeEngine Module: This module consists of one submodule:
 - a. MemeEngine → This is implemented in MemeEngine.py. This module defines MemeGenerator class. Output directory location must be supplied while creating object

of type MemeGenerator. This class is basically responsible for taking input image and quote, turn it into meme and return path to the saved meme. Following methods are associated with this class:

- i. `load_image` method → This method is responsible for opening supplied image file and return Image Object reference. If there is any problem loading the image file, it throws exception and returns None.
 - ii. `transform_image` method → This method takes Image object , resizes it and then returns resized/transformed Image object
 - iii. `add_caption` method → This method takes transformed image and file extension as input and then draws quote over transformed image turning it into meme, saves it and returns the saved image path.
 - iv. `make_meme` method → This is the master method which is invoked by `app.py` and `meme.py` to generate meme. It takes input image, quote body and quote author and then calls `load_image`, `transform_image` and `add_caption` method in order specified and returns the filepath for saved meme to calling program.
- 3) Meme Command Line Module → This is a CLI program implemented in `meme.py` that can be invoked directly by user. It takes three optional arguments → Image File Path, Quote Body and Quote Author. It calls and implements `generate_meme` function which takes Image File Path, Quote Body and Quote Author as input. If these values are not specified(optional), then it parses locally stored files to randomly select an image and quote and then call `make_meme` method implemented by MemeGenerator class to generate meme and get the path to saved meme file which is then returned as output to the user. This module makes use of MemeGenerator, Ingestor and QuoteModel class modules.
- 4) Web Meme Generation Interface Module → This module is implemented in `app.py`. Logic implemented in module is somewhat similar to that of `meme.py`. It implements function by name of `setup()` which helps in parsing and randomly selecting local image file and Quotes stored in local files. This function is used when user selects the option to randomly generate meme in web interface. If user selects Create option in web interface, he/she needs to provide a valid URL from which image file can be loaded, Quote Body and Quote Author. Once the meme is generated, it is displayed back in web interface. It makes use of flask server to serve the web interface. This module makes use of Ingestor, QuoteModel and MemeGenerator classes to implement the functionality.

How to use the command line interface

- 1) Run without arguments:

```
(newnenv) (base) PS C:\Users\shahe\python_udacity\meme-generator-starter-code\src> python .\meme.py
[< "Treat yo self", Fluffles >, < "Life is like a box of treats", Forrest Pup >, < "It's the size of the fight in the do
g", Bark Twain >]
The path of generated meme is ./tmp/img_155722.jpg
```

Generated Meme



2) Run with all arguments supplied:

```
(newnenv) (base) PS C:\Users\shahe\python_udacity\meme-generator-starter-code\src> python .\meme.py -h
usage: meme.py [-h] [--path PATH] [--body BODY] [--author AUTHOR]

Please enter Image File Path, Quote and Author

optional arguments:
  -h, --help            show this help message and exit
  --path PATH            Image File Path Including File Name
  --body BODY            Quote which needs to be displayed in Meme
  --author AUTHOR        Author of the Quote
(newnenv) (base) PS C:\Users\shahe\python_udacity\meme-generator-starter-code\src> python .\meme.py --path "./PIC.jpg" --bo
dy "I failed my way to success" --author "EDISON"
The path of generated meme is ./tmp/img_360028.jpg
```

Generated Meme:



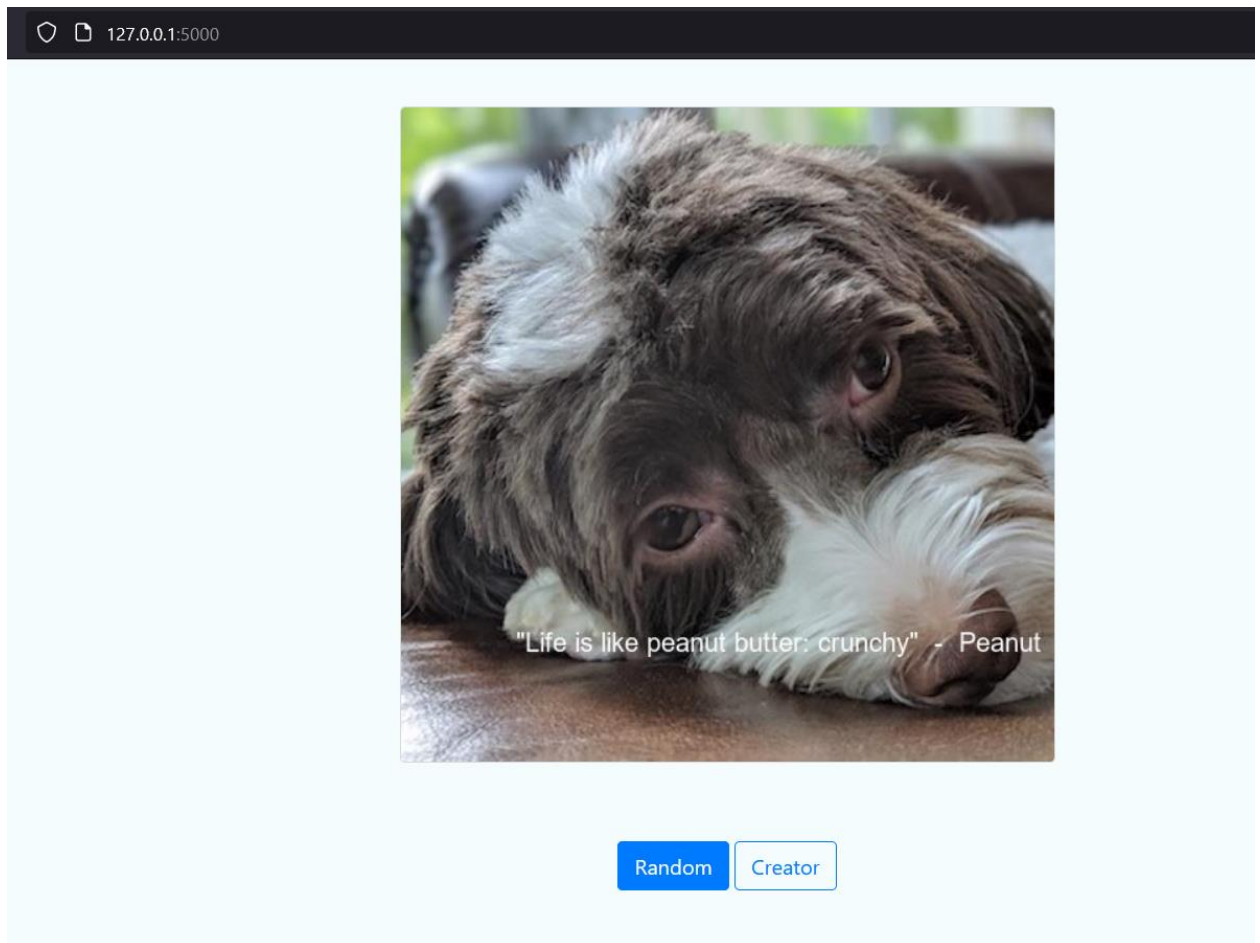
How to use Web Interface

1) Start the flask server:

```
(newnenv) (base) PS C:\Users\shahe\python_udacity\meme-generator-starter-code\src> python .\app.py
[< "Treat yo self", Fluffles >, < "Life is like a box of treats", Forrest Pup >, < "It's the size of the fight in the do
g", Bark Twain >]
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

2) Goto url <http://127.0.0.1:5000>

Randomly generated meme is displayed. Click on Random to generate more random memes



```
Path generated is ./static/img_863119.jpg
127.0.0.1 - - [07/Dec/2022 21:24:59] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [07/Dec/2022 21:24:59] "GET /static/img_863119.jpg HTTP/1.1" 200 -
127.0.0.1 - - [07/Dec/2022 21:24:59] "GET /favicon.ico HTTP/1.1" 404 -
Path generated is ./static/img_179251.jpg
127.0.0.1 - - [07/Dec/2022 21:26:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [07/Dec/2022 21:26:20] "GET /static/img_179251.jpg HTTP/1.1" 200 -
127.0.0.1 - - [07/Dec/2022 21:26:20] "GET /favicon.ico HTTP/1.1" 404 -
```



Random

Creator

- 3) To create Meme by supplying your own image file, quote body and quote author, click on Creator and fill in values in respective text boxes. Then hit on "Create Meme"

You can test it with this image url :

<https://images.pexels.com/photos/753626/pexels-photo-753626.jpeg>

Image URL

<https://images.pexels.com/photos/753626/pexels-photo-753626.jpeg>

Quote Body

If you can dream it, you can do it !!

Quote Author

Walt Disney

Create Meme!

Random

Creator



Random

Creator

XPDF is installed in sw folder:

```
(newnenv) (base) PS C:\Users\shahe\python_udacity\meme-generator-starter-code\src> ls sw

Directory: C:\Users\shahe\python_udacity\meme-generator-starter-code\src\sw


Mode                LastWriteTime         Length Name
----                -
d-----          05-12-2022    21:22             xpdf-tools-win-4.04

(newnenv) (base) PS C:\Users\shahe\python_udacity\meme-generator-starter-code\src>
```

Path to pdftotext set in PATH variable

```
PATH=C:\Users\shahe\python_udacity\meme-generator-starter-code\src\sw\xpdf-tools-win-4.04\bin64;
```